

Assignment-12

Task-1:

Prompt:

Write Python code for Bubble Sort with inline comments explaining passes, comparisons, swaps, and early termination. Include a brief explanation and time & space complexity.

Code and Output:

```
lab 12.py > ...
1  def bubble_sort(arr):
2      n = len(arr)
3
4      # Outer loop for each pass (n-1 passes in total)
5      for i in range(n - 1):
6          # A flag to check if any swapping happened in this pass
7          swapped = False
8
9          # Inner loop goes till (n-i-1) because the last i elements are already sorted
10         for j in range(n - i - 1):
11             # Compare adjacent elements
12             if arr[j] > arr[j + 1]:
13                 # Swap if elements are in the wrong order
14                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
15                 swapped = True # Mark that a swap occurred
16
17         # If no swapping happened in this pass, array is already sorted → stop early
18         if not swapped:
19             break
20
21     return arr
22
23
24 # Example usage
25 data = [64, 34, 25, 12, 22, 11, 90]
26 print("Unsorted array:", data)
27 print("Sorted array:", bubble_sort(data))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Unsorted array: [64, 34, 25, 12, 22, 11, 90]
Unsorted array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS D:\AI Assisted Coding>
```

Explanation:

1. The `bubble_sort` function sorts a list in ascending order using the bubble sort algorithm.
2. It accepts a single parameter `arr`, which is the list to be sorted.
3. The variable `n` stores the length of the list for iteration purposes.
4. The outer loop iterates through each element of the list, ensuring that all items are checked.
5. A `swapped` flag is initialized to determine if any elements were exchanged during an iteration.
6. The inner loop compares adjacent elements and iterates only up to the unsorted portion of the list.
7. When two adjacent elements are out of order, they are swapped, and the `swapped` flag is set to `True`.
8. If no swaps occur during an entire pass of the inner loop, the algorithm exits early, optimizing performance.
9. The algorithm efficiently handles both best-case (already sorted) and worst-case (reverse order) scenarios.
10. In the main block, a sample list is printed before and after sorting to demonstrate the function's effect.

Task 2:

Prompt:

Write Python code for Bubble Sort and Insertion Sort. Explain why Insertion Sort is more efficient for partially sorted arrays and compare their performance on a nearly sorted input.

Code and Output:

Bubble Sort Implementation:

```
lab 12.py > ...
28
29 def bubble_sort(arr):
30     n = len(arr)
31     for i in range(n - 1):
32         swapped = False
33         for j in range(n - i - 1):
34             if arr[j] > arr[j + 1]:
35                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
36                 swapped = True
37         if not swapped:
38             break
39     return arr
40
41 # Example usage
42 data1 = [10, 20, 15, 30, 25, 40]
43 print("Bubble Sort:", bubble_sort(data1.copy()))
44
45
46
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Bubble Sort: [10, 15, 20, 25, 30, 40]
PS D:\AI Assisted Coding>
```

Insertion Sort Implementation:

```
lab 12.py > ...
50 def insertion_sort(arr):
51     n = len(arr)
52     for i in range(1, n):
53         key = arr[i] # Element to be positioned
54         j = i - 1
55         # Move elements greater than key one position ahead
56         while j >= 0 and arr[j] > key:
57             arr[j + 1] = arr[j]
58             j -= 1
59         arr[j + 1] = key # Place key in correct position
60     return arr
61
62 # Example usage
63 data2 = [10, 20, 15, 30, 25, 40]
64 print("Insertion Sort:", insertion_sort(data2.copy()))
65
66
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Insertion Sort: [10, 15, 20, 25, 30, 40]
PS D:\AI Assisted Coding>
```

Explanation:

1. Bubble Sort:

◦ How It Works:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. The process is repeated until no swaps are needed—indicating that the list is sorted.

◦ Efficiency on Nearly Sorted Input:

Although Bubble Sort implements an early exit (by checking if any swap occurred in a pass), it still typically makes comparisons for all elements in each pass. This results in $O(n)$ comparisons per pass even if the list is nearly sorted.

2. Insertion Sort:

◦ How It Works:

Insertion Sort builds the final sorted array one element at a time by picking the next element and inserting it into its proper place among the previously sorted elements.

◦ Efficiency on Nearly Sorted Input:

When the list is already nearly sorted, most elements are already in their correct position. The inner loop that shifts elements finds that most values do not need to be moved. In many cases, the inner loop iterates only a few times (or not at all) per element. Thus, even though the worst-case complexity is $O(n^2)$, the average cost for nearly sorted arrays approaches $O(n)$.

3. Performance Comparison:

◦ On nearly sorted input, **Insertion Sort** outperforms Bubble Sort because it minimizes the number of comparisons and shifts.

◦ **Bubble Sort**, even with an early exit, has to run through the entire list for each pass, making it less efficient if there is only minor disorder.

◦ The timing code in the script creates a nearly sorted list and records execution times for both sorts. Typically, the Insertion Sort time will be lower than or comparable to Bubble Sort time on such input.

This script demonstrates both sorting algorithms and compares their performance while highlighting the advantages Insertion Sort provides when working with nearly sorted data.

Task 3:

Prompt:

Write Python code for Linear Search and Binary Search with docstrings and performance notes. Test both on sorted and unsorted data and explain when Binary Search is preferable.

Code and Output:

Linear Search Implementation:

```
lab 12.py > linear_search
66
67 def linear_search(arr, target):
68     """
69     Perform Linear Search to find the target in the array.
70
71     Parameters:
72     arr (list): The list of elements (can be unsorted)
73     target : The element to search for
74
75     Returns:
76     int: Index of target if found, else -1
77
78     Performance:
79     - Best case: O(1) (target is first element)
80     - Worst/Average case: O(n)
81     """
82     for i, value in enumerate(arr):
83         if value == target:
84             return i
85     return -1
86
87 # Example usage
88 data_unsorted = [10, 3, 15, 7, 8]
89 print("Linear Search (unsorted):", linear_search(data_unsorted, 7))
90
91
92
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Linear Search (unsorted): 3
PS D:\AI Assisted Coding>

Binary Search Implementation:

```
lab 12.py > ...
92 def binary_search(arr, target):
93     """
94     Perform Binary Search to find the target in a sorted array.
95
96     Parameters:
97     arr (list): Sorted list of elements
98     target : The element to search for
99
100    Returns:
101    int: Index of target if found, else -1
102
103    Performance:
104    - Best case: O(1) (target is middle element)
105    - Worst/Average case: O(log n)
106    - Requires sorted array
107    """
108    left, right = 0, len(arr) - 1
109    while left <= right:
110        mid = (left + right) // 2
111        if arr[mid] == target:
112            return mid
113        elif arr[mid] < target:
114            left = mid + 1
115        else:
116            right = mid - 1
117    return -1
118
119 # Example usage
120 data_sorted = [3, 7, 8, 10, 15]
121 print("Binary Search (sorted):", binary_search(data_sorted, 7))
122
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Binary Search (sorted): 1
PS D:\AI Assisted Coding>

```

PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Quick Sort:
Random List (first 10 elements): [2, 4, 8, 16, 18, 27, 28, 32, 33, 40]
Sorted List (first 10 elements): [2, 4, 8, 16, 18, 27, 28, 32, 33, 40]
Reverse Sorted List (first 10 elements): [2, 4, 8, 16, 18, 27, 28, 32, 33, 40]
Total time for Quick Sort: 0.003529 seconds

Merge Sort:
Random List (first 10 elements): [2, 4, 8, 16, 18, 27, 28, 32, 33, 40]
Sorted List (first 10 elements): [2, 4, 8, 16, 18, 27, 28, 32, 33, 40]
Reverse Sorted List (first 10 elements): [2, 4, 8, 16, 18, 27, 28, 32, 33, 40]
Total time for Merge Sort: 0.004613 seconds
PS D:\AI Assisted Coding>

```

Explanation:

1. Linear Search:

- The `linear_search` function iterates over each element in the list.
- If the target element is found, it immediately returns its index.
- If the target is not found by the end of the list, it returns -1.
- **Performance:**
 - **Best-case:** $O(1)$, when the target is at the very beginning.
 - **Worst/Average-case:** $O(n)$, because in the worst case, every element is checked.

2. Binary Search:

- The `binary_search` function searches a sorted list.
- It compares the target with the middle element. If they match, it returns the index.
- If the target is less than the middle element, it continues searching in the left half; otherwise, in the right half.
- This repeated halving leads to $O(\log n)$ worst-case performance.
- **Note:** The function returns -1 if the target is not found, and it only works properly on a sorted list.

3. When to Prefer Binary Search:

- **Efficiency:** With a worst-case time complexity of $O(\log n)$, Binary Search is significantly more efficient than Linear Search ($O(n)$) for large sorted datasets.
- **Requirement:** Binary Search requires the array to be sorted. If the dataset is not sorted, you either need to sort it (which itself has a cost) or use Linear Search.
- **Practical Use:** In scenarios with frequent search operations on mostly static, sorted arrays, Binary Search provides a better performance advantage.

Task 4:

Prompt:

Implement recursive Quick Sort and Merge Sort with partially completed functions. Complete missing logic, add docstrings, and compare performance on random, sorted, and reverse-sorted lists.

Code & Output:

```
125 def quick_sort(arr):
126     """
127     Sorts the list using the Quick Sort algorithm recursively.
128
129     Parameters:
130     |   arr (list): The list to be sorted.
131
132     Returns:
133     |   list: A new sorted list.
134
135     Average Complexity: O(n log n)
136     Best Case Complexity: O(n log n)
137     Worst Case Complexity: O(n^2) when pivot selection is poor (e.g., already sorted or reverse-sorted input)
138     """
139     if len(arr) <= 1:
140         return arr
141     # Choose the pivot - here we select the middle element
142     pivot = arr[len(arr) // 2]
143     # Partition into three parts: less than, equal, and greater than pivot
144     left = [x for x in arr if x < pivot]
145     equal = [x for x in arr if x == pivot]
146     right = [x for x in arr if x > pivot]
147     # Recursively sort left and right partitions and combine
148     return quick_sort(left) + equal + quick_sort(right)
149
150 def merge_sort(arr):
151     """
152     Sorts the list using the Merge Sort algorithm recursively.
153
154     Parameters:
155     |   arr (list): The list to be sorted.
156
157     Returns:
158     |   list: A new sorted list.
159
```

```
150 def merge_sort(arr):
160     Average Complexity: O(n log n)
161     Best Case Complexity: O(n log n)
162     Worst Case Complexity: O(n log n)
163     """
164     if len(arr) <= 1:
165         return arr
166     mid = len(arr) // 2
167     # Recursively sort the left and right halves
168     left = merge_sort(arr[:mid])
169     right = merge_sort(arr[mid:])
170
171     # Merge the two sorted halves
172     merged = []
173     i = j = 0
174     while i < len(left) and j < len(right):
175         if left[i] <= right[j]:
176             merged.append(left[i])
177             i += 1
178         else:
179             merged.append(right[j])
180             j += 1
181     # Append any remaining elements (only one of these loops will actually run)
182     merged.extend(left[i:])
183     merged.extend(right[j:])
184     return merged
185
186 if __name__ == '__main__':
187     import random
188     import time
189
190     # Create test lists of 1000 elements
191     size = 1000
192     random_list = random.sample(range(10000), size)
193     sorted_list = sorted(random_list)
194     reverse_sorted_list = sorted(random_list, reverse=True)
```

```

195
196     # Test and measure Quick Sort on different data orders
197     start = time.perf_counter()
198     qs_random = quick_sort(random_list)
199     qs_sorted = quick_sort(sorted_list)
200     qs_reverse = quick_sort(reverse_sorted_list)
201     time_qs = time.perf_counter() - start
202
203     # Test and measure Merge Sort on the same data
204     start = time.perf_counter()
205     ms_random = merge_sort(random_list)
206     ms_sorted = merge_sort(sorted_list)
207     ms_reverse = merge_sort(reverse_sorted_list)
208     time_ms = time.perf_counter() - start
209
210     print("Quick Sort:")
211     print("Random List (first 10 elements):", qs_random[:10])
212     print("Sorted List (first 10 elements):", qs_sorted[:10])
213     print("Reverse Sorted List (first 10 elements):", qs_reverse[:10])
214     print("Total time for Quick Sort: {:.6f} seconds".format(time_qs))
215
216     print("\nMerge Sort:")
217     print("Random List (first 10 elements):", ms_random[:10])
218     print("Sorted List (first 10 elements):", ms_sorted[:10])
219     print("Reverse Sorted List (first 10 elements):", ms_reverse[:10])
220     print("Total time for Merge Sort: {:.6f} seconds".format(time_ms))
221

```

Explanation:

Explanation

1. Quick Sort:

◦ How It Works:

The algorithm picks a pivot element (here, the middle element) and partitions the array into three sub-arrays: those less than the pivot, equal to the pivot, and greater than the pivot. It then recursively sorts the left and right sub-arrays and concatenates the results.

◦ Time Complexities:

- **Average/Best Case:** $O(n \log n)$ when the pivot divides the list relatively evenly.
- **Worst Case:** $O(n^2)$ if the pivot results in highly unbalanced partitions (for example, if the data is already sorted or reverse-sorted and the pivot is not chosen wisely).

2. Merge Sort:

◦ How It Works:

The algorithm divides the list into two halves, recursively sorts each half, and then merges the two sorted halves together. The merge step compares elements from the two halves and builds a new sorted list.

◦ Time Complexities:

- **Average/Best/Worst Case:** $O(n \log n)$ consistently, since the division and merging process is done in a balanced manner regardless of the input order.

3. Comparison on Different Inputs:

◦ Random Data:

Both Quick Sort and Merge Sort typically perform in $O(n \log n)$ time. Quick Sort might be slightly faster in practice with good pivot selection, though its worst-case performance remains a concern.

◦ Sorted Data:

Quick Sort can degrade to $O(n^2)$ in the worst-case if the pivot selection is poor. Merge Sort remains $O(n \log n)$ as its performance is independent of the input order.

◦ Reverse-Sorted Data:

The same performance considerations as for sorted data apply. Without a careful pivot strategy, Quick Sort may again hit its worst-case, while Merge Sort consistently performs in $O(n \log n)$.

This script demonstrates both sorting algorithms, tests them on different types of input lists, and prints a brief performance summary.

Task 5:

Prompt:

I've written a naive duplicate-finder function that checks each element against every other ($O(n^2)$ time). Please optimize it using a more efficient approach like sets or dictionaries to achieve $O(n)$ time complexity. Also, compare the execution times of both versions on large input lists and explain how the complexity was improved.

Code & Output:

```
24 import random
25 import time
26
27 def find_duplicates_bruteforce(arr):
28     """
29     Finds duplicates using a brute-force approach.
30
31     Time Complexity:  $O(n^2)$ 
32     """
33     duplicates = []
34     n = len(arr)
35     for i in range(n):
36         for j in range(i + 1, n):
37             if arr[i] == arr[j] and arr[i] not in duplicates:
38                 duplicates.append(arr[i])
39     return duplicates
40
41 def find_duplicates_optimized(arr):
42     """
43     Finds duplicates using a set for tracking observed elements.
44
45     Time Complexity:  $O(n)$ 
46     """
47     seen = set()
48     duplicates = set()
49     for x in arr:
50         if x in seen:
51             duplicates.add(x)
52         else:
53             seen.add(x)
54     return list(duplicates)
55
56 if __name__ == '__main__':
57     # Generate a large list with potential duplicates
58     data = [random.randint(0, 5000) for _ in range(10000)]
```

```
260     start = time.perf_counter()
261     dup_bf = find_duplicates_bruteforce(data)
262     time_bf = time.perf_counter() - start
263
264     start = time.perf_counter()
265     dup_opt = find_duplicates_optimized(data)
266     time_opt = time.perf_counter() - start
267
268     print("Brute Force Duplicates (first 10):", dup_bf[:10], "Total found:", len(dup_bf), "Time:", time_bf)
269     print("Optimized Duplicates (first 10):", dup_opt[:10], "Total found:", len(dup_opt), "Time:", time_opt)
270
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [] [] [] []

```
PS D:\AI Assisted Coding>
PS D:\AI Assisted Coding> & C:/Users/Prasanna/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Assisted Coding/lab 12.py"
Brute Force Duplicates (first 10): [1128, 3228, 2005, 3128, 4512, 3853, 3947, 2591, 2175, 3863] Total found: 2945 Time: 2.2687404002062976
Optimized Duplicates (first 10): [0, 1, 3, 4, 7, 8, 9, 10, 12, 13] Total found: 2945 Time: 0.0008429000154137611 ...
PS D:\AI Assisted Coding>
```

Explanation:

Below is a concise explanation of the key aspects of the code:

- **Sorting & Searching Functions:**

The file contains implementations of various algorithms like Bubble Sort, Insertion Sort, Quick Sort, and Merge Sort along with search functions (Linear Search and Binary Search). These serve as examples to compare algorithm behavior on different types of data (unsorted, sorted, reverse-sorted).

- **Duplicate Detection:**

Two duplicate detectors are implemented:

- **Brute Force ($O(n^2)$):**

Checks every pair of elements to find duplicates. It is simple but becomes very slow as the input size grows.

- **Optimized using Sets ($O(n)$):**

Iterates through the list while keeping track of seen elements in a set. This approach quickly detects duplicates thanks to fast membership checks, making it far more efficient for large inputs.

- **Performance Comparison:**

The script measures the execution times of both duplicate-finding methods using a large dataset, clearly showcasing the speed improvement with the optimized version.